مكتب

Lab Activity

Week 9: List Algorithms - Search, Sort, and Comprehensio

Duration: 60 minutes — Work in pairs!

Name: _____ **Partner:** _____

Lab Goals

By the end of this lab, you will be able to:

- Implement linear search to find items in lists
- Use sort() and sorted() appropriately
- Create lists efficiently with list comprehensions
- Apply accumulator, filter, and search patterns
- Avoid common pitfalls when modifying lists

Getting Started

- Extract the given `week9_lab.zip` file to your desktop
- Remember: sort() modifies in place, sorted() creates new list!
- List comprehensions: `[expression for item in list if condition]`

1 Exercise 1: Linear Search Implementation (10 minutes)

Master the fundamental search algorithm!

Open `week9_ex1.py` and complete:

```
# Linear search - find first occurrence
def find_item(my_list, target):
    """Returns index of target, or -1 if not found"""
    for i in range(len(my_list)):
        if my_list[i] == target:
            return ___  # Return index when found

    return ___  # Return -1 if not found

# Test the function
scores = [85, 92, 78, 95, 88, 92]
print("Scores:", scores)

# Find specific scores
result = find_item(scores, 78)
if result != -1:
    print(f"Found 78 at index {result}")
else:
    print("78 not found")
```

```
20
21 # Find all occurrences
22 def find_all(my_list, target):
23     """Returns list of all indices where target appears"""
24     positions = []
25     for i in range(len(my_list)):
26         if my_list[i] == target:
27             positions._____(i)  # Add index to positions
28
29     return positions
30
31 # Test find_all
32 all_92s = find_all(scores, 92)
33 print(f"92 appears at indices: {all_92s}")
```

Tip: Linear search checks each item one by one - simple but works on any list!

Checkpoint: Search functions working? Show teacher!

2 Exercise 2: Sorting Mastery (10 minutes)

Learn when to use sort() vs sorted()!

Open week9_ex2.py and complete:

```
1  # Method 1: sort() - modifies original
2  numbers1 = [3, 1, 4, 1, 5, 9, 2, 6]
3  print("Original:", numbers1)
4
5  numbers1.___()  # Sort in place
6  print("After sort():", numbers1)
7  print("Original was modified!")
8
9  # Method 2: sorted() - creates new list
10 numbers2 = [3, 1, 4, 1, 5, 9, 2, 6]
11 print("\nOriginal:", numbers2)
12
13 sorted_nums = _____(numbers2)  # Creates new sorted list
14 print("Sorted version:", sorted_nums)
15 print("Original unchanged:", numbers2)
16
17 # Reverse sorting
18 numbers3 = [78, 92, 65, 88, 95]
19 numbers3.sort(reverse=___)  # Sort descending
20 print("\nDescending:", numbers3)
21
22 # Custom sorting - by length
23 words = ["cat", "elephant", "dog", "ant"]
24 words.sort(key=___)  # Sort by length
25 print("By length:", words)
26
27 # Alphabetical (case-insensitive)
28 names = ["sara", "Ali", "ahmed", "Fatima"]
29 names.sort(key=str.lower)  # Ignore case
30 print("Alphabetical:", names)
```

Pair Programming: Switch who types every 5 minutes!

3 Exercise 3: List Comprehensions (10 minutes)

Create lists elegantly in one line!

Open week9_ex3.py and complete:

```python
# Basic comprehension - transform
numbers = [1, 2, 3, 4, 5]

# Traditional way
squares = []
for num in numbers:
    squares.append(num ** 2)

# Comprehension way
squares = [num ** 2 for num in numbers]
print("Squares:", squares)

# With condition - filter and transform
evens_doubled = [num * 2 for num in numbers if num % 2 == 0]
print("Evens doubled:", evens_doubled)

# Practice: Complete these comprehensions
grades = [75, 82, 91, 78, 95, 88, 67]

# Get all passing grades (>= 60)
passing = [g for g in grades if ___________]
print("Passing:", passing)

# Get all A grades (>= 90)
a_grades = [___ for ___ in ___ if ___]
print("A grades:", a_grades)

# Add 5 bonus points to all grades
with_bonus = [g + ___ for g in grades]
print("With bonus:", with_bonus)

# String comprehensions
words = ["hello", "world", "python"]
uppercase = [word.___() for word in words]
print("Uppercase:", uppercase)

# Get words longer than 5 characters
long_words = [___ for ___ in ___ if len(___) > 5]
print("Long words:", long_words)
```

Tip: List comprehensions are great for simple transformations. Use regular loops for complex logic!

Checkpoint: Comprehensions mastered? Excellent!

4 Exercise 4: Algorithm Patterns (10 minutes)

Apply common patterns to solve problems!

Open week9_ex4.py and complete:

```python
# Pattern 1: Accumulator
def calculate_total(prices):
    """Sum all prices"""
    total = 0
    for price in prices:
        total += price
    return total

prices = [100, 250, 75, 320]
print(f"Total: Rs.{calculate_total(prices)}")

# Pattern 2: Find with condition
def find_first_passing(grades):
    """Find first grade >= 60"""
    for grade in grades:
        if grade >= ___:
            return grade
    return None  # None if not found

grades = [45, 52, 68, 82, 55]
first_pass = find_first_passing(grades)
print(f"First passing grade: {first_pass}")

# Pattern 3: MinMax with position
def find_hottest_day(temperatures):
    """Returns (day_number, temperature) of hottest day"""
    hottest_day = 0
    for i in range(len(temperatures)):
        if temperatures[i] > temperatures[hottest_day]:
            hottest_day = i

    return (hottest_day + 1, temperatures[hottest_day])

temps = [28, 32, 29, 35, 31, 33, 30]
day, temp = find_hottest_day(temps)
print(f"Hottest: Day {day} with {temp} C")

# Pattern 4: Filter
def get_high_scores(scores, threshold=85):
    """Returns all scores >= threshold"""
    high_scores = []
    for score in scores:
        if _______________:  # Fill condition
            high_scores._____(_____)

    return high_scores

scores = [85, 92, 78, 95, 88, 72, 90, 85]
high = get_high_scores(scores)
print(f"High scores (85+): {high}")
```

5 Exercise 5: Advanced Search and Filter (10 minutes)

Combine techniques for more complex problems!

Open week9_ex5.py and complete:

```python
# Student grade analysis
students = ["Ali", "Sara", "Ahmed", "Fatima", "Hassan"]
grades = [85, 92, 78, 95, 88]

# Find student with highest grade
def find_top_student(students, grades):
    """Returns name of student with highest grade"""
    top_index = 0
    for i in range(len(grades)):
        if grades[i] > grades[top_index]:
            top_index = i

    return students[top_index]

top = find_top_student(students, grades)
print(f"Top student: {top}")

# Get students who passed (>= 60)
def get_passing_students(students, grades, threshold=60):
    """Returns list of (name, grade) for passing students"""
    passing = []
    for i in range(len(students)):
        if grades[i] >= threshold:
            passing.append((students[i], grades[i]))

    return passing

passing = get_passing_students(students, grades)
print("\nPassing students:")
for name, grade in passing:
    print(f"  {name}: {grade}")

# Find all students with grade in range
def find_in_range(students, grades, min_grade, max_grade):
    """Returns students with grades between min and max"""
    result = []
    for i in range(len(students)):
        if ___ <= grades[i] <= ___:  # Fill in
            result.append((students[i], grades[i]))

    return result

b_students = find_in_range(students, grades, 80, 89)
print("\nB grade students (80-89):")
for name, grade in b_students:
    print(f"  {name}: {grade}")
```

Checkpoint: Complex searches working? Great job!

6 Exercise 6: Avoiding Pitfalls (10 minutes)

Learn to safely modify lists during iteration!

Open week9_ex6.py and complete:

```python
# Pitfall 1: Removing while iterating (WRONG)
print("=== WRONG WAY ===")
numbers = [1, 2, 3, 2, 4, 2, 5]
print("Original:", numbers)

# This will MISS some items!
for num in numbers:
    if num == 2:
        numbers.remove(num)

print("After removal:", numbers)  # Still has a 2!
print("BUG: Items were skipped!")

# Fix 1: Iterate over copy
print("\n=== FIX 1: Copy ===")
numbers = [1, 2, 3, 2, 4, 2, 5]
for num in numbers.___():  # Iterate over copy
    if num == 2:
        numbers.remove(num)

print("After removal:", numbers)  # All 2's removed!

# Fix 2: Use list comprehension (best)
print("\n=== FIX 2: Comprehension ===")
numbers = [1, 2, 3, 2, 4, 2, 5]
numbers = [num for num in numbers if num != ___]
print("After removal:", numbers)

# Pitfall 2: Modifying indices while iterating
print("\n=== SAFE: Iterate backwards for removal ===")
grades = [45, 72, 68, 55, 82, 90, 38, 75]
print("Original:", grades)

# Remove failing grades (<60) - iterate backwards!
for i in range(len(grades) - 1, -1, -1):
    if grades[i] < ___:
        grades._____(i)  # Remove by index

print("After removing failing:", grades)

# Practice: Remove all vowels from words
words = ["apple", "banana", "orange"]
vowels = "aeiou"

# Use comprehension to keep only non-vowels
cleaned = []
for word in words:
    clean_word = ''.join([c for c in word if c.lower() not in vowels])
    cleaned.append(clean_word)

print("\nOriginal words:", words)
print("Without vowels:", cleaned)
```

Tip: When removing items: use comprehension, iterate over copy, or go backwards!

Checkpoint: Pitfalls avoided? You're a pro!

7 Lab Summary

Checkpoint: Final checkpoint - make sure you have:

- ☐ Implemented linear search algorithms
- ☐ Used sort() and sorted() correctly
- ☐ Created lists with comprehensions
- ☐ Applied algorithm patterns (accumulator, find, filter)
- ☐ Solved complex search problems
- ☐ Avoided iteration modification pitfalls
- ☐ Tested all code with various inputs
- ☐ Worked effectively with your partner

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	Need Help	Getting It	Got It!
Linear search	○	○	○
Sorting (sort vs sorted)	○	○	○
List comprehensions	○	○	○
Safe list modification	○	○	○

Which algorithm pattern is most useful for your projects?

_____

Why is iterating over a copy safer when removing items?

_____

Excellent! You're now an algorithms expert!